Warehouse Management System

Technical Architecture, Entity-Relationship (ER) Specification & Technology Stack

MYSQL 8.0.46 INNODB | NODE.JS 20.18 + EXPRESS | REACT 18 + TYPESCRIPT | TAILWIND CSS V4

ZERO CLOUD DEPENDENCY / 100% OFFLINE | ACID COMPLIANT

ARCHITECTURE PATTERN	TOTAL DATABASE TABLES	FOREIGN KEY RELATIONS	TARGET ENVIRONMENT
Modular REST + SPA Tier	20 InnoDB Relational Tables	44 Referential Constraints	macOS Native / Standalone

1. System Architecture & Operational Capabilities

The Warehouse Stock & Shipment Management System is an enterprise-grade warehouse execution and logistics control platform designed according to clean, restrained human-centered engineering principles. The platform eliminates extraneous visual clutter in favor of maximum operational density, sub-second query response, and strict transactional consistency.

The application operates completely offline on local hardware without recurring internet calls or SaaS lock-in, with local persistence stored in an InnoDB binary database structure.

1. **Master Catalog & Multi-Warehouse Bin Topology**
   Hierarchical mapping of commercial inventory across regional facilities, zones, aisles, racks, and discrete storage bins with custom barcodes.
2. **Inbound ASN Intake & Putaway Processing**
   Structured vendor purchase order receipts verifying manifests, recording physical counts, logging damaged units, and assigning destination bins.
3. **Order Allocation & Concurrency-Safe Reservations**
   Automated reservation locks prevent over-promising and race conditions by isolating available stock from committed inventory.
4. **Outbound Pick, Pack, Dispatch & Carrier Tracking**
   Multi-stage outbound fulfillment pipeline with pick lists, package verification, tracking numbers, and atomic physical deductions upon dispatch.
5. **Security, Role-Based Access & Tamper-Evident Auditing**
   Four-tier role permissions (Admin, Manager, Staff, Viewer), password salting with bcrypt, and immutable audit logs capturing every critical state change.

2. Technologies Used & Architecture Matrix

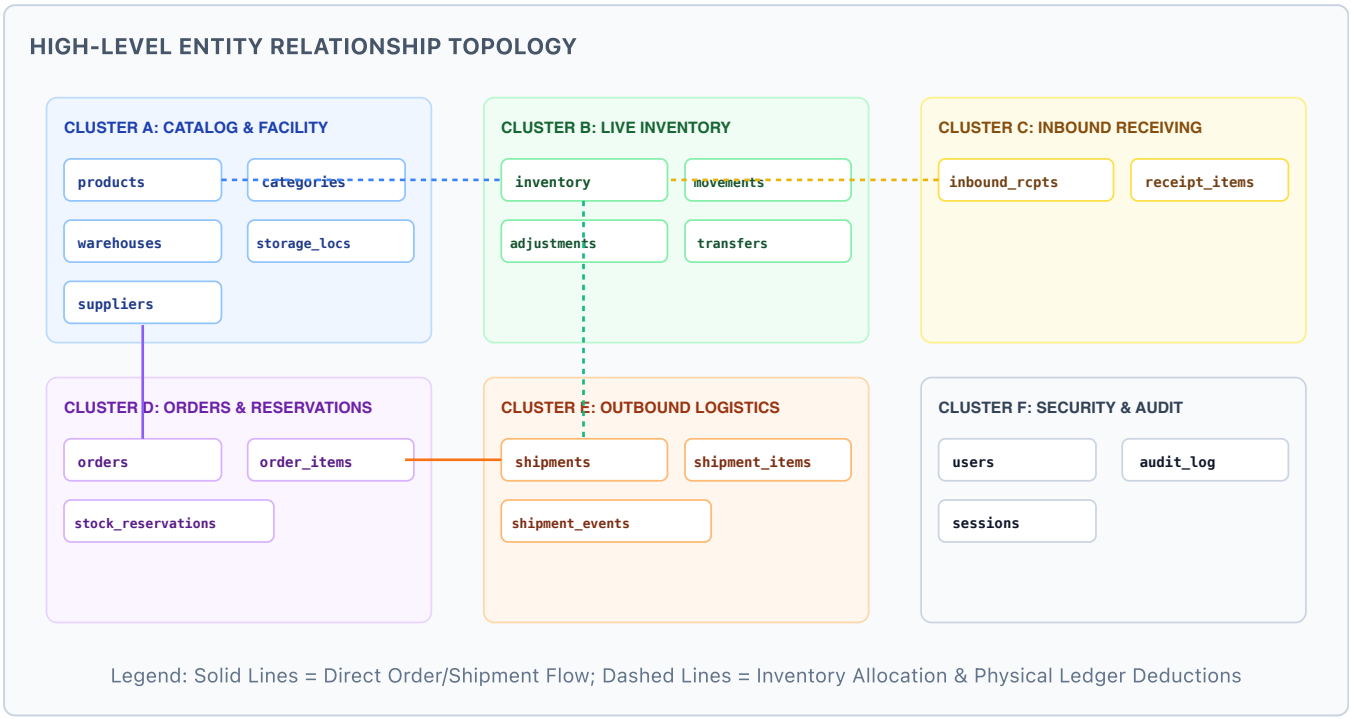
The following matrix details all foundational libraries, frameworks, database systems, and runtime utilities employed across the backend, frontend, database, and automation layers.

TIER / DOMAIN	TECHNOLOGY & VERSION	PACKAGE / BINARY	ARCHITECTURAL PURPOSE & IMPLEMENTATION DETAIL
Primary Database	**MySQL Community Server** `8.0.46`	`mysqld` binary	High-performance relational engine running **InnoDB**. Enforces ACID guarantees, row-level locks (`SELECT ... FOR UPDATE`), foreign key referential integrity, and persistent local data storage.
Database Client	**mysql2 / promise** `^3.9.7`	`mysql2`	Asynchronous Promise-based connection pooling with connection recycling, parameterized prepared statements against SQL injection, and programmatic transaction lifecycle control.
Server Runtime	**Node.js LTS** `v20.18.0`	`node`	Local high-throughput event loop executing backend microservices without external cloud dependencies.
Web Application Framework	**Express.js** `^4.18.2`	`express`	RESTful routing architecture organizing modular controllers for authentication, catalog, inventory, receiving, orders, shipments, suppliers, audit logs, and analytics.
Programming Language	**TypeScript** `^5.6.2`	`typescript`	Strict end-to-end type safety shared between API schemas, database entity types, and frontend React components.
Authentication & Sessions	**JWT + Express-Session** `jsonwebtoken 9.0.2`	`express-session`	Hybrid stateful/stateless authentication providing Bearer token headers for API interactions and HTTP-only cookie sessions for browser navigation.
Password Cryptography	**bcryptjs** `^2.4.3`	`bcryptjs`	10-round salted adaptive password hashing guaranteeing cryptographically secure credential persistence in the `users` table.
HTTP Security & CORS	**Helmet & CORS** `helmet 7.1 / cors 2.8`	`helmet` , `cors`	Hardened HTTP response headers (Content-Security-Policy, HSTS, X-Frame-Options) and strict origin validation restricting unauthorized client access.
UI Framework	**React** `18.3.1`	`react` , `react-dom`	Declarative component architecture with custom hooks, unified `AuthContext` , and high-frequency state updates.

TIER / DOMAIN	TECHNOLOGY & VERSION	PACKAGE / BINARY	ARCHITECTURAL PURPOSE & IMPLEMENTATION DETAIL
Build Tool & Bundler	**Vite** `5.4.21`	`vite`	Lightning-fast Rollup-based asset bundler with ES-module Hot Module Replacement (HMR) and optimized tree-shaken static compilation.
Design System & Styling	**Tailwind CSS v4** `4.3.3`	`@tailwindcss/vite`	Google product aesthetic: restrained slate palette, 1px subtle borders, precise typographic scale, compact table layouts, and responsive grids.
Client-Side Routing	**React Router DOM** `7.18.4`	`react-router-dom`	Client-side route guards, authenticated redirects, deep-linking, and URL search parameter synchronization.
Iconography	**Heroicons v2** `2.2.0`	`@heroicons/react`	Pure vector SVG line icons maintaining visual elegance and semantic clarity without decorative emoji.
Data Visualization	**Recharts** `3.10.1`	`recharts`	SVG-rendered operational dashboards tracking daily stock movements, inventory thresholds, and shipment statuses.
Offline Automation Scripts	**macOS Command & Shell** `POSIX / ZSH`	`WarehouseManager.command`	Single-click native desktop launcher orchestrating MySQL daemon startup, Express server instantiation, Vite frontend serving, and automatic Chrome browser launch.

3. Database Entity-Relationship (ER) Architecture

The database consists of **20 InnoDB tables** organized into five interconnected functional clusters. All primary keys are auto-incrementing integers, and referential integrity is strictly enforced via 44 foreign key constraints with explicit `CASCADE` or `SET NULL` behaviors.



4. Comprehensive Database Data Dictionary (20 Tables)

The detailed technical specifications below describe all 20 InnoDB tables, indexing strategies, nullability rules, and domain-level business constraints.

1. users — SECURITY

PK	id	INT AUTO_INCREMENT
UQ	username	VARCHAR(100) NOT NULL
UQ	email	VARCHAR(191) NOT NULL
	password_hash	VARCHAR(255) NOT NULL
	full_name	VARCHAR(255) NOT NULL
	role	ENUM('admin','manager','staff','viewer')
	is_active	TINYINT DEFAULT 1
	created_at	TIMESTAMP DEFAULT NOW

2. categories — MASTER DATA

PK	id	INT AUTO_INCREMENT
UQ	name	VARCHAR(100) NOT NULL
	description	TEXT NULL
	created_at	TIMESTAMP DEFAULT NOW

3. warehouses — FACILITIES

PK	id	INT AUTO_INCREMENT
UQ	name	VARCHAR(150) NOT NULL
UQ	code	VARCHAR(50) NOT NULL
	address	VARCHAR(255) NULL
	city / state	VARCHAR(100) NULL
	capacity	INT DEFAULT 10000
	is_active	TINYINT DEFAULT 1

4. storage_locations — FACILITIES

PK	id	INT AUTO_INCREMENT
FK	warehouse_id	INT NOT NULL (warehouses.id)
	zone / aisle	VARCHAR(50) NULL
	rack / bin	VARCHAR(50) NULL
UQ	label	VARCHAR(100) NOT NULL
	is_active	TINYINT DEFAULT 1

5. suppliers — PROCUREMENT

PK	id	INT AUTO_INCREMENT
	name	VARCHAR(150) NOT NULL
UQ	code	VARCHAR(50) NOT NULL
	contact_person	VARCHAR(150) NULL
	email / phone	VARCHAR(191) / VARCHAR(50)
	is_active	TINYINT DEFAULT 1

6. products — CATALOG

PK	id	INT AUTO_INCREMENT
UQ	sku	VARCHAR(100) NOT NULL
	name	VARCHAR(255) NOT NULL
FK	category_id	INT NULL (categories.id)
UQ	barcode	VARCHAR(100) NULL
	unit_cost	DECIMAL(10,2) DEFAULT 0.00
	reorder_threshold	INT DEFAULT 10

7. inventory — LIVE STOCK

PK	id	INT AUTO_INCREMENT
FK	product_id	INT NOT NULL (products.id)
FK	warehouse_id	INT NOT NULL (warehouses.id)
FK	location_id	INT NULL (storage_locations.id)
	quantity_on_hand	INT NOT NULL DEFAULT 0
	reserved_quantity	INT NOT NULL DEFAULT 0
	reorder_threshold	INT DEFAULT 10
UQ	Composite Key	(product_id, warehouse_id, location_id)

8. inventory_movements — LEDGER

PK	id	INT AUTO_INCREMENT
FK	product_id	INT NOT NULL (products.id)
FK	warehouse_id	INT NOT NULL (warehouses.id)
FK	location_id	INT NULL (storage_locations.id)
	movement_type	ENUM('receive','ship','adjust','transfer_
	quantity	INT NOT NULL
FK	performed_by	INT NULL (users.id)

9. orders — ORDERS

PK	id	INT AUTO_INCREMENT
UQ	order_number	VARCHAR(100) NOT NULL
	customer_name	VARCHAR(255) NOT NULL
	status	ENUM('pending','confirmed','processing','partial
	priority	ENUM('low','normal','high','urgent')
FK	created_by	INT NULL (users.id)
	order_date	TIMESTAMP DEFAULT NOW

10. order_items — ORDERS

PK	id	INT AUTO_INCREMENT
FK	order_id	INT NOT NULL (orders.id CASCADE)
FK	product_id	INT NOT NULL (products.id)
	quantity_ordered	INT NOT NULL
	quantity_fulfilled	INT NOT NULL DEFAULT 0
FK	warehouse_id	INT NULL (warehouses.id)
	unit_price	DECIMAL(10,2) DEFAULT 0.00

11. stock_reservations — ALLOCATION

PK	id	INT AUTO_INCREMENT
FK	order_id	INT NOT NULL (orders.id)
FK	order_item_id	INT NOT NULL (order_items.id)
FK	product_id	INT NOT NULL (products.id)
FK	warehouse_id	INT NOT NULL (warehouses.id)
FK	location_id	INT NULL (storage_locations.id)
	quantity	INT NOT NULL
	status	ENUM('active','fulfilled','released')

12. inbound_receipts — INBOUND

PK	id	INT AUTO_INCREMENT
UQ	receipt_number	VARCHAR(100) NOT NULL
FK	supplier_id	INT NULL (suppliers.id)
FK	warehouse_id	INT NOT NULL (warehouses.id)
	status	ENUM('draft','in_progress','completed','cancelle
	expected_date / received	DATE NULL
FK	created_by	INT NULL (users.id)

13. receipt_items — INBOUND

PK	id	INT AUTO_INCREMENT
FK	receipt_id	INT NOT NULL (inbound_receipts.id)
FK	product_id	INT NOT NULL (products.id)
	expected_quantity	INT NOT NULL DEFAULT 0
	received_quantity	INT NOT NULL DEFAULT 0
	damaged_quantity	INT NOT NULL DEFAULT 0
FK	location_id	INT NULL (storage_locations.id)

14. shipments — OUTBOUND

PK	id	INT AUTO_INCREMENT
UQ	shipment_number	VARCHAR(100) NOT NULL
FK	order_id	INT NULL (orders.id)
FK	warehouse_id	INT NOT NULL (warehouses.id)
	status	ENUM('draft','ready_to_pick','picking','packed',
	carrier / tracking	VARCHAR(100) NULL
FK	assigned_to	INT NULL (users.id)

15. shipment_items — OUTBOUND

PK	id	INT AUTO_INCREMENT
FK	shipment_id	INT NOT NULL (shipments.id)
FK	product_id	INT NOT NULL (products.id)
	quantity_ordered	INT NOT NULL
	quantity_picked	INT NOT NULL DEFAULT 0
	quantity_packed	INT NOT NULL DEFAULT 0
FK	location_id	INT NULL (storage_locations.id)

16. shipment_status_events — OUTBOUND

PK	id	INT AUTO_INCREMENT
FK	shipment_id	INT NOT NULL (shipments.id)
	status	VARCHAR(50) NOT NULL
	notes	TEXT NULL
FK	created_by	INT NULL (users.id)
	created_at	TIMESTAMP DEFAULT NOW

17. stock_adjustments — STOCK CONTROL

PK	id	INT AUTO_INCREMENT
FK	product_id	INT NOT NULL (products.id)
FK	warehouse_id	INT NOT NULL (warehouses.id)
	adjustment_type	ENUM('count','damage','loss','found','c
	qty_before / after	INT NOT NULL
FK	performed_by	INT NULL (users.id)

18. inventory_transfers — TRANSFERS

PK	id	INT AUTO_INCREMENT
FK	product_id	INT NOT NULL (products.id)
FK	from_wh / to_wh	INT NOT NULL (warehouses.id)
	quantity	INT NOT NULL
	status	ENUM('pending','completed','cancelled')
FK	performed_by	INT NULL (users.id)

19. audit_log — AUDITING

PK	id	INT AUTO_INCREMENT
FK	user_id	INT NULL (users.id)
	action	VARCHAR(100) NOT NULL
	entity_type / entity_id	VARCHAR(100) / INT
	details	JSON NULL
	ip_address	VARCHAR(100) NULL
	created_at	TIMESTAMP DEFAULT NOW

20. sessions — SECURITY

PK	sid	VARCHAR(128)
	sess	JSON NOT NULL
IDX	expired	BIGINT NOT NULL

5. Concurrency Protection & Transaction Lifecycle

Warehouse operations run concurrently across multiple staff members picking, packing, and dispatching orders. To prevent double dispatch, ghost inventory, or negative stock balances, all state-changing endpoints execute under strict MySQL InnoDB transactions with exclusive row-level locking.

Atomic Shipment Dispatch Execution Flow

1. **Transaction Initialization**
   `START TRANSACTION` acquired on dedicated MySQL connection.
2. **Pessimistic Row Lock on Shipment**
   Executes `SELECT * FROM shipments WHERE id = ? FOR UPDATE`. Any rival request blocks until lock resolution.
3. **Idempotency Validation**
   Checks if status is already `dispatched` or `delivered`. If true, triggers immediate `ROLLBACK` and responds with `HTTP 409 Conflict` to prevent duplicate dispatch.
4. **Pessimistic Inventory Row Locks & Availability Check**
   Executes `SELECT * FROM inventory WHERE product_id = ? AND warehouse_id = ? FOR UPDATE`. Validates that `quantity_on_hand >= shipped_qty`.
5. **Physical Balance Deductions & Reservation Clearance**
   Decrements `quantity_on_hand -= shipped_qty` and releases corresponding `reserved_quantity`. Updates `stock_reservations` to `fulfilled`.
6. **Audit Trail & Movement Ledger Logging**
   Inserts immutable row in `inventory_movements` with movement type `'ship'` and logs user action in `audit_log`.
7. **Atomic Commit**
   `COMMIT` executed. All table changes become durable simultaneously; locks are released.

6. Offline Deployment Architecture & Port Bindings

The system is configured to launch and operate completely disconnected from external internet infrastructure. A single executable command script coordinates the 3-process architecture:

COMPONENT	BINDING / PROTOCOL	PROCESS DETAILS
MySQL 8.0 Daemon	`localhost:3306` & `/tmp/mysql.sock`	Standalone local daemon storing tables in `data/mysql`. Managed via background process runner.
Express REST API	`localhost:3001`	Node.js API server handling authentication, validation, queries, and transactions.
React Vite Client	`localhost:5173`	Vite development/preview server serving the single-page application with proxying to port 3001.
Launcher Automation	`WarehouseManager.command`	Native macOS executable located on Desktop and project root that initializes services and opens Google Chrome automatically.